

Note: Do not edit this. download the template and start working.



Prompt Log

Team Name	Dietcoke
Website	https://akashavani-mauve.vercel.app/
Domain	Travel
GitHub	https://github.com/N9601/Skyscanner-Rebuild

GUIDELINES:

1. Log all significant AI prompts used during development.
2. Mention the purpose of each prompt and the AI tool used.
3. Record important refinements, outputs, and modifications.
4. Do not fabricate or alter entries after submission.
5. Continue the log throughout the entire development process.

It's not just about the code, but how you use AI to build it.

Instructions to Participants:

- Recreate the website as a functional and interactive web experience, not just a static visual representation.
- AI tools may be used as development assistants for generating code, debugging, improving UI, implementing functionality, and solving technical problems.
- Prompts must be based on the team's own observations and analysis from Round 1.
- Follow all technical and resource restrictions specified by the organizers.
- Participants must not explicitly reveal the identity of the target website to AI.

Better prompts. Smarter builds.

Prompt	AI Tools Used	Output Received	How It was used
Handover: rebuild target = Akashavani (never name the real site). Read the prompt log, clear old entries, and start fresh from this prompt as entry #1. Follow hard rules: log every significant prompt in the PDF, append-only, no fabrication, ground prompts in the team's own Round-1 decode observations.	Claude Code	Read the existing RE_BUILD-PromptLog.pdf which contained six planning-phase entries (feasibility study, PRD, tech stack, handover doc). Built a Python + reportlab pipeline that regenerates the PDF from a JSON entries file while faithfully preserving the official template: RE:BUILD purple wordmark, DECODE. REBUILD. EVOLVE tagline, Prompt Log heading, Team/Website/Domain header table, the five guidelines, participant instructions, and the 'Better prompts. Smarter builds.' footer, followed by the four-column entries table (Prompt, AI Tools Used, Output Received, How It was used) on page two onward. Reset the table so this handover became row 1.	Established the reproducible logging pipeline used for every subsequent prompt in this document, and gave the Round-2 build a clean, append-only audit trail from its first instruction.
Scaffold the Vite project, do an initial commit and push, and create the React app basic components. Stack per TechStack.md (React 18 + TS + Vite, Tailwind, React Router v6, TanStack Query, Zustand, lucide-react). Repo: N9601/Skyscanner-Rebuild.	Claude Code	Hand-wrote a complete 38-file scaffold at the repo root instead of running the interactive Vite CLI: package.json with pinned dependencies, vite.config.ts with an @ path alias, strict tsconfig pair, tailwind.config.ts with a custom brand palette (brand blue, surface, ink, eco tokens), dark-mode class strategy, custom shadows and container settings, PostCSS config, index.html, ESLint/Prettier configs, and .gitignore. Source tree: main.tsx entry, AppProviders with a tuned TanStack QueryClient, createBrowserRouter with routes for Home, Flights, Stays, Cars, Trips, Alerts, Assistant and a 404, Layout/Header/Footer shell, Button and Card primitives, a SearchWidget with trip-type tabs, and styled stub pages for every surface. Initialized git on main, added the GitHub remote, verified the remote was empty, committed and pushed.	This became the working baseline application; every later feature (mock APIs, filters, stores, assistant) was layered onto this structure without reworking it.

Prompt	AI Tools Used	Output Received	How It was used
UI inspiration screenshots from Dribbble shared (Booking Adventures hero, travel dashboard with rings, Vita Travels dark chip quiz, Voyage 3D carousel, Adventuro stat overlays); use framer-motion; repo remade, push again as initial commit with N9601 as the only contributor in everything.	Claude Code	Analyzed the five reference shots and extracted a design vocabulary to build toward: big friendly display type with one accent-colored word, glassy segmented search bar with a circular dark submit button, black pill CTAs, floating 3D-style decor elements, dark cinematic destination carousel with tilted side cards, progress rings and lime accents for dashboards, and chip-based quiz interactions. Deleted the local .git history, re-initialized, and pushed a single clean initial commit authored solely by N9601 with no AI co-author trailers. Installed framer-motion and committed it separately as the animation foundation.	The extracted visual language drove every subsequent UI decision (hero, search bar, carousel, trips dashboard, assistant chips), and the repo history now shows a single human contributor as required.
Make commits on every small change we do.	Claude Code	Switched to a granular commit discipline: every logical unit of work (a dependency addition, the design-token pass, the data layer, each page, each fix) became its own descriptively-messaged commit pushed immediately to origin/main. The final history contains 40+ commits telling the build story step by step, from 'chore: add framer-motion' through 'feat(flights): whole-month price heatmap' to 'fix(tour): continuously re-track spotlight target'.	Gave the judges an auditable, replayable history that maps one-to-one onto the prompts in this log, and made every regression trivially bisectable during the time-pressured final hours.

Prompt	AI Tools Used	Output Received	How It was used
UI must feel like a proper premium SaaS app, no em dashes, build the app fully end to end.	Claude Code	Executed the full MVP in one sustained pass of small commits: self-hosted Inter and Manrope via Fontsource for offline safety; expanded design tokens (10-step brand scale, lifted/glow shadows, hero grid background); persisted Zustand stores for theme, trip plan and price alerts; premium sticky header with a spring-animated nav pill (framer-motion layoutId), dark-mode toggle and live trip-count badge; the segmented glass search bar with keyboard-navigable airport autocomplete; Flights results with a filter rail (stops, price slider, airline checkboxes, Greener Choice switch), four sort pills, a 9-day price calendar strip, and animated result cards with add-to-trip and create-alert actions; Stays and Cars result surfaces; a Trips dashboard with animated SVG count rings, an editable budget bar and a lime gradient total card; an Alerts manager with triggered states; and a scripted travel-only assistant with an intent router covering routes, budgets, vibes, greener queries and polite refusals. All copy written without em dashes.	Every surface promised in the PRD became demonstrable in the browser, turning the scaffold into a complete product for the demo.

Prompt	AI Tools Used	Output Received	How It was used
Fill the app with local and international flights using actual flight data; do we need any APIs?	Claude Code	Concluded no external API is needed and would conflict with the offline-demo constraint (live APIs like Amadeus/Duffel need keys, network and quota). Instead built a realistic data core: a dataset of 36 real airports (DEL, BOM, BLR through DXB, SIN, LHR, JFK, SYD) with true IATA codes, official airport names and latitude/longitude; a roster of 21 real airlines (IndiGo, Air India, Vistara, Emirates, Qatar Airways, Singapore Airlines, Lufthansa, Qantas and more) with domestic/long-haul capability flags. Flight generation now computes great-circle distance via the haversine formula, derives duration from realistic cruise speeds, prices per kilometre with separate domestic, regional and long-haul curves and floors (BLR-GOI around 3-5k INR, DEL-LHR around 35-60k INR), routes stopovers through plausible hubs (DEL/BOM/BLR domestically, DXB/DOH/IST/SIN internationally), and estimates CO2 per distance. A seeded PRNG keyed on the route makes every search deterministic and stable across reloads.	Search results for any city pair, domestic or international, now look and price like the real world while running fully offline on the demo laptop.
UI seems bland; show an actual SaaS UI taking heavy inspiration from the shared Dribbble shots.	Claude Code	Rebuilt the key surfaces to match the reference language directly: the search widget became a single elevated glass bar with labelled segments divided by hairlines and a circular dark search button (the Booking Adventures signature); the hero gained a soft blue gradient canvas, grid pattern, floating plane chip, boarding-pass card and CO2 pill decorations with slow float animations; primary CTAs became black pills; and a new dark 'Trending escapes' section renders a 3D perspective carousel where the centered card is full size and side cards rotate away in space (the Voyage signature), with arrow navigation, click-to-focus, hashtag chips and a glass info pill showing tagline, from-price and an Explore link.	Home and search stopped reading as a template and started reading as the premium travel product the reference shots depict.

Prompt	AI Tools Used	Output Received	How It was used
Take animation and styling inspiration from my profaxis project.	Claude Code	Read the profaxis codebase (globals.css, motion components, landing page) and ported its motion language into Akashavani as reusable pieces: an IntersectionObserver-based Reveal that fades content up as it enters the viewport, a cursor-following TiltCard with a moving radial sheen, a CountUp that rolls numbers with cubic easing and en-IN formatting, a letterpress line-rise hero reveal where headline lines emerge from clipped masks, route-fade page entrances, a barely-there SVG paper-grain film over the whole app, an infinite ticker-tape marquee with edge masking, hairline card borders that sharpen with a shadow pop on hover, and prefers-reduced-motion guards on every animation.	The whole app now shares one coherent, editorial-grade motion system; the fare ticker, stat count-ups, scroll reveals and hero reveal all trace back to this pass.
I have a Gemini API key for the assistant chat; wire it up and make a .env file to paste the key.	Claude Code	Created a gitignored .env (verified git refuses to stage it) and a committed .env.example, then wired a Gemini client that reads VITE_GEMINI_API_KEY and calls generateContent with a travel-only system prompt enforcing concision, INR pricing, refusal of non-travel topics and a no-em-dash style. The assistant became a hybrid: deterministic intents that produce action buttons (route, budget, vibe) stay on the instant scripted router, while open-ended questions go to Gemini with the last eight turns of history; any API failure silently falls back to the scripted reply so the demo can never break. The panel header and footer switch between 'Gemini powered' and 'Scripted demo' automatically.	The assistant gained real LLM conversation when the key is present while remaining fully functional offline, exactly matching the venue-risk mitigation in the feasibility study.

Prompt	AI Tools Used	Output Received	How It was used
Append the prompt log; fix overlapping containers; add actual airline logos, HD photos with animation on trending escapes; other pages are too empty, add data.	Claude Code	Four fixes in one pass: flight cards now render real airline logos from the kiwi.com logo CDN keyed by IATA code, with a colored monogram fallback if the image fails; carousel destination cards gained Unsplash photography with a slow hover zoom and the gradient kept underneath as a fallback layer; the carousel's info pill moved out of the card stage into its own row below, ending the overlap with card titles; and the empty no-query states of Flights, Stays and Cars were filled with a Popular Routes grid (eight real city pairs with from-prices linking straight into searches) and Popular Cities photo tiles. Also caught and fixed a real routing bug found while testing in the browser: 'Dubai (DXB)' style input was not parsed by findAirport, silently falling back to Delhi; the parser now extracts the IATA code from parentheses first.	Every landing surface now carries content, results show authentic airline branding, and international searches resolve to the correct destination.
Make an actual 2D SVG logo, current one seems basic.	Claude Code	Designed a custom mark in hand-written SVG: a rounded-square badge with a blue gradient, an orbit ring, a dashed contrail arc, and a layered paper plane built from three facets in white and light blues. Shipped as a React Logo component used in the header and footer, plus a standalone favicon.svg, replacing the generic lucide compass icon.	Gave Akashavani an ownable brand mark across the app shell and the browser tab.
Move the globe left so nothing is over it.	Claude Code	Repositioned the hero's gradient globe decoration from the center of the illustration column to 30 percent from the left, so the floating boarding-pass card and CO2 pill sit clear of it instead of overlapping.	The hero illustration cluster now reads as an arranged composition rather than stacked elements.

Prompt	AI Tools Used	Output Received	How It was used
Change the photos, add something iconic and beautiful.	Claude Code	Curated landmark photography for the carousel and verified every candidate Unsplash URL returned HTTP 200 before committing: Maldives overwater villas, a Kashmir lake for Srinagar, the Burj Khalifa skyline for Dubai, Wat Arun at golden hour for Bangkok, and the Chureito Pagoda with Mount Fuji for Tokyo, keeping the Goa palm beach and the Eiffel Tower shot. Each image keeps its gradient underlay and an onError hide so a failed load can never leave a broken card.	The Trending Escapes carousel now sells each destination with instantly recognizable imagery.
Needs a better, more minimalistic UI (assistant screenshot shared).	Claude Code	Redesigned the assistant from a heavy dark-teal panel into a minimal hairline card that matches the rest of the app: compact chat bubbles (ink-on-white for the user, quiet muted surface for the assistant), suggestion chips shown only before the first message, a rounded borderless-feeling input with a monochrome circular send button, three subtle pulsing typing dots, and a centered page header that names whether Gemini or the scripted engine is answering.	The assistant stopped competing visually with the product and settled into the app's minimalist premium language.
What major features and UI should come next?	Claude Code	Proposed a ranked roadmap with reasoning: 1) a flight detail drawer because results felt shallow without a detail layer, 2) Everywhere search as the signature meta-search moment, 3) a whole-month price grid extending the day strip, 4) a lazy-loaded MapLibre stays map for visual payoff, 5) a polish pass covering a Ctrl+K command palette, upgraded page transitions and a mobile bottom nav.	Became the literal build order for the remainder of the hackathon; the team picked the sequence 5, 4, 3, 1 next.
Is Gemini connected?	Claude Code	Tested the key live with curl rather than trusting the UI badge: the key authenticated, but gemini-2.5-flash returned 404 with a notice that it is no longer available to new API users. Switched the client to gemini-3.6-flash, re-ran the test, confirmed a 200 with a correct completion, and committed the one-line model fix.	Caught a silent production breakage before the demo; the assistant's live path is verified working end to end.

Prompt	AI Tools Used	Output Received	How It was used
Build the roadmap in this order: 5, 4, 3, 1.	Claude Code	Shipped all four as separate commits. Polish: a Ctrl+K command palette (fuzzy filter over pages, popular routes and a dark-mode action, full keyboard navigation), animated route transitions, and a fixed mobile bottom nav with safe-area padding. Stays map: lazy-loaded MapLibre GL with free OSM raster tiles, price-chip markers (■6k style pills) that highlight and scroll to their card on click, fitBounds framing, and a list/map split layout; stay data gained deterministic coordinates jittered around each city's real airport location. Month grid: a fetchMonthPrices generator with weekend uplift, rendered as a calendar heatmap colored by price tercile with the cheapest day ringed in green, expanding under the day strip and driving the search date on click. Detail drawer: a spring slide-over with airline logo, a vertical segment timeline including layover city, distance and duration chips, a fare breakdown (base, taxes, fees), and animated CO2 bars comparing the flight against the route average, plus save and alert actions. Fixed an esModuleInterop/maplibre typing issue found by the type-checker along the way.	These four features carry the core of the judging demo: depth on flights, a map wow moment, and speed polish.
Fix UI, don't leave any cards empty across the site, use proper photos (screenshot showed the autocomplete dropdown fighting the city cards).	Claude Code	Diagnosed the screenshot: the airport autocomplete dropdown was being overlapped by the Popular Cities cards below, so the search widget received its own elevated stacking context (relative z-30) putting the dropdown cleanly above all following content. Then eliminated flat gradient cards: Popular City tiles gained verified Unsplash photos of each city with hover zoom and a darkening gradient for text legibility, and all nine stay cards per city now cycle through a curated, URL-verified pool of hotel and resort photographs layered over their gradients.	No card on the site renders as a flat color block anymore, and the search dropdown is always readable.

Prompt	AI Tools Used	Output Received	How It was used
Clipping text (hero screenshot shared).	Claude Code	The letterpress line-rise mask (overflow: hidden) was cutting the descenders of 'j' and 'y' in the headline word 'journey'. Fixed with the standard compensation: 0.18em of padding-bottom on the mask cancelled by an equal negative margin, so descenders render fully while the rise animation still clips correctly.	The hero headline renders pixel-perfect with the reveal animation intact.
What next to make it a fully fledged app?	Claude Code	Mapped the gap from demo to product: Everywhere search, a simulated booking flow to close the dead-end after results, round-trip depth, Supabase auth with cloud sync, recent searches, detail pages for stays and cars, a live alert loop, assistant deep links, deployment, tests and a quality audit. Recommended the cut: Everywhere, booking, auth, then ship-quality.	Defined the closing scope; the team greenlit it with 'seal the deal' and supplied Supabase credentials.

Prompt	AI Tools Used	Output Received	How It was used
Fix nav badge centering and seal the deal; UI still bland, needs over-the-top animation and less white; Supabase URL and publishable key shared.	Claude Code	Large closing pass. Centered the trips badge glyph with flex centering and leading-none. Everywhere search: leaving the destination empty now routes to an Explore grid that prices every tracked airport from the origin via the haversine engine, sorted ascending, with domestic/international scope chips and budget caps. Booking simulation: a three-stage flow (contacting airline spinner, gradient fare-review card with base/taxes/fees and a demo-safety notice, then a spring-animated handoff screen with a booking reference), reachable from a new Book button in the detail drawer. Visual richness: a fixed aurora layer of three slow-drifting blurred color orbs behind every page, an animated gradient on the hero accent word, gradient count-up stats in a tinted panel, a tinted ticker strip, brand-glow card hovers, gradient primary buttons with a shine sweep on hover, and a gradient scroll-progress bar. Supabase: verified the project's auth health endpoint returned 200, installed supabase-js, added a guarded client, an auth store on onAuthStateChanged, a sign-in/sign-up modal with confirm-email handling, a header account menu, debounced cloud sync of trips and alerts into a user_data table, and a ready-to-run RLS schema script.	The app gained its last signature feature, a complete booking story, real accounts with cloud sync, and a far richer visual identity in one stretch.
Fast, last 10 minutes.	Claude Code	Compressed the ship-quality tail into minutes: added vitest (pinned to a Vite-5-compatible major), wrote 13 tests covering mock API determinism, airport parsing from 'City (IATA)' strings, greener-flag correctness, international price floors, month-grid cheapest-day invariants, Everywhere sort order, formatters and the intent router's routes, budgets, vibes and refusals; all 13 passed first run. Added the Vercel SPA rewrite config, committed, pushed, and brought this log current.	The submission gained a test suite and one-click deployability inside the deadline window.

Prompt	AI Tools Used	Output Received	How It was used
App deployed to Vercel; fill the Website field in the prompt log with the live URL. Also this stretch: blank /book after drawer navigation fixed, faster booking load, handoff links to real airline sites, in-site onboarding tour, hand-drawn logo, profile page, photos on every explore card.	Claude Code	Set Website to https://akashavani-mauve.vercel.app/ in the log header. The referenced fixes, each its own commit: the page-transition exit phase (AnimatePresence mode wait) was deadlocking when navigating from the open drawer leaving /book blank until reload, removed the exit phase so pages always mount instantly; booking's connecting stage cut from 1.4s to 0.6s; the handoff screen's provider button now opens the real airline website (21 airlines mapped: goindigo.in, emirates.com, qatarairways.com and more) in a new tab; a four-step first-visit spotlight tour (search, hub nav, assistant, Ctrl+K) with a dimmed cutout, progress dots, skip/next, scroll lock, localStorage persistence and a bottom-sheet layout on mobile; the logo rebuilt as a hand-drawn ink doodle (wobbly stroked paper plane, loopy dotted blue contrail) after feedback that the gradient badge felt AI-generated; a Profile page with a gradient identity header, count-up stats for trip items, planned spend, alerts and estimated CO2 saved, theme toggle, sign-out and quick links; and every Explore card gained a photo via a 24-city verified Unsplash map with a rotating verified fallback pool.	The log header now points at the live submission, and the final UX rough edges were closed before recording.

Prompt	AI Tools Used	Output Received	How It was used
Wire the Gemini API deeply: extract structured intent so replies carry real in-app actions. Finalize everything.	Claude Code	Upgraded the Gemini path from plain text to structured extraction: the request now sends a responseSchema asking for JSON of { reply, action { kind: flights/stays/cars/explore/none, from, to, city, depart } } with action rules in the system prompt (default origin Bengaluru, dates only when the user gave one). The app resolves the returned cities through the real airport dataset and renders a live search button on the reply, so 'I want to see Tokyo in November' yields a tappable BLR to NRT search. Parsing is defensive: it extracts the first JSON object even if the model wraps it in prose, degrades to a plain text reply if parsing fails, and falls back to the scripted router on any API error (verified against a live 503 spike during testing). Finalized with a clean type-check, 13/13 tests passing, a production build, and pushed everything.	Natural language now drives real app navigation, completing the assistant as a functional copilot rather than a chat box.
Post-finalization polish: fix the walkthrough tour misalignment, add the GitHub link to this log, drop model names from the tools column, make the log detailed, and support the video walkthrough recording.	Claude Code	The tour's spotlight was measured once and drifted when fonts, images and entry animations shifted the layout underneath it (a screenshot showed the ring over the stats panel instead of the search bar); it now re-measures the target element every 180ms while open, so the highlight stays glued through any layout shift. Added a GitHub row to this log's header table pointing at the repository. Normalized the AI Tools column to 'Claude Code'. Rewrote every row of this log in full detail at the team's request, expanding the condensed summaries into complete accounts of what was built, while keeping the append-only, no-fabrication rule: nothing here describes work that did not happen. Produced a timed 2:30 shot list with narration for the demo video and a Vercel environment-variable checklist (the live site needs the three VITE_ vars set and a redeploy for sign-in, Profile and Gemini to appear, since Vite bakes env at build time).	The tour is demo-safe, the log is a complete detailed record with repo and live links, and the team has a ready script for the video submission.

Better prompts. Smarter builds.